

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Курсовой проект
по курсу «Технологии параллельного программирования»

Обратная трассировка лучей (Ray Tracing) на GPU.

Выполнил: *Серый Никита Олегович*

Группа: *М8О-409Б-22*

Преподаватель: *А.Ю. Морозов.*

Москва, 2025

Условие

Использование GPU для создание фотореалистической визуализации. Рендеринг полужеркальных и полупрозрачных правильных геометрических тел. Получение эффекта бесконечности. Создание анимации.

Сцена. Прямоугольная текстурированная поверхность (пол), над которой расположены три платоновых тела. Сверху находятся несколько источников света. На каждом ребре многогранника располагается определенное количество точечных источников света. Грани тел обладают зеркальным и прозрачным эффектом. За счет многократного переотражения лучей внутри тела, возникает эффект бесконечности.

Камера. Камера выполняет облет сцены согласно определенным законам. В цилиндрических координатах (r , ϕ , z), положение и точка направления камеры в момент времени t определяется следующим образом:

1. Цель работы: Требуется реализовать алгоритм обратной трассировки лучей (<http://www.ray-tracing.ru/>) с использованием технологии CUDA. Выполнить пок кадровый рендеринг сцены. Для устранения эффекта «зубчатости», выполнить сглаживание (например с помощью алгоритма SSAA). Полученный набор кадров склеить в анимацию любым доступным программным обеспечением. Подобрать параметры сцены, камеры и освещения таким образом, чтобы получить наиболее красочный результат. Провести сравнение производительности `gri` и `cpi` (т.е. дополнительно нужно реализовать алгоритм без использования CUDA).
2. Вариант 6. Тетраэдр, Додекаэдр, Икосаэдр.

Пример входных данных:

1024

/home/checker/pgp/ivanov/kp/out/img_%d.data

640 480 120

7.0 3.0 0.0 2.0 1.0 2.0 6.0 1.0 0.0 0.0

2.0 0.0 0.0 0.5 0.1 1.0 4.0 1.0 0.0 0.0

2.0 0.0 0.0 1.0 0.0 0.0 1.0 0.9 0.1 10

0.0 2.0 0.0 0.0 1.0 0.0 0.75 0.8 0.2 5

0.0 0.0 0.0 0.0 0.7 0.7 0.5 0.7 0.3 2

-5.0 -5.0 -1.0 -5.0 5.0 -1.0 5.0 5.0 -1.0 5.0 -5.0 -1.0 ~/floor.data 0.0 1.0 0.0 0.5

2

-10.0 0.0 10.0 1.0 1.0 1.0

1.0 0.0 10.0 0.0 0.0 1.0

10 16

Программа должна поддерживать следующие ключи запуска:

--cpu Для расчетов используется только центральный процессор

--gpu Для расчетов задействуется видеокарта

--default В stdout выводится конфигурация входных данных (в формате описанном ранее) при которой получается наиболее красочный результат, после чего программа завершает свою работу.

Запуск программы без аргументов подразумевает запуск с ключом --gpu.

В процессе работы программа должна выводить в stdout статистику в формате:

```
{номер кадра}\t{время на обработку кадра в миллисекундах}\t{общее количество лучей}\n
```

Программное и аппаратное обеспечение

Графический процессор (Google Colab)

Compute capability	7.5
Name	Tesla T4
Total Global Memory	15828320256
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)
Total constant memory	6553
Multiprocessors count	40

Процессор AMD Ryzen 5 5500U (2.1 GHz, max 4 GHz)

Technology	7 нм
Cores	6
Threads	12
Cache (Level 2)	64К (на ядро)
Cache (Level 2)	512К (на ядро)
Cache (Level 3)	8 МБ (всего)

Оперативная память

Number of modules	2
Module Size	8 ГБ
Frequency	3200 МГц
Generation	DDR4

Жесткий диск - 512 ГБ SSD

OS - Ubuntu 22

IDE - VS Code, theme Monokai; gedit, theme Oblivion

Compiler - nvcc

Метод решения

Общее описание алгоритма решения задачи, архитектуры программы и т. п. Полностью расписывать алгоритмы необязательно, но в общих чертах описать нужно. Приветствуются ссылки на внешние источники, использованные при подготовке (книги, интернет-ресурсы).

Алгоритм обратной трассировки лучей

Программа реализует классический алгоритм обратной трассировки: лучи пускаются не от источников света, а **от глаза (камеры) через пиксели воображаемого экрана в сцену**.

1. **Генерация луча:** Для каждого пикселя вычисляются координаты в пространстве камеры, которые затем переводятся в мировые координаты с помощью базиса камеры (ex,ey,ez).
2. **Пересечение (Intersection):** Луч проверяется на пересечение со всеми треугольниками в сцене (tracePrimary). Используется алгоритм нахождения пересечения луча с плоскостью треугольника.
3. **Затенение (Shading):**
 - Если луч не пересекает ни один объект, пикселю присваивается цвет неба.
 - Если пересечение найдено, вычисляется освещенность в точке попадания. Программа проверяет наличие тени (isShadowed), пуская дополнительный луч от точки пересечения к источнику света. Если на пути этого луча встречается объект, точка считается затененной.

Сглаживание

Для борьбы с эффектом «лесенок» на краях объектов применен метод **Super Sampling**. Вместо того чтобы пускать один луч через центр пикселя, программа делит пиксель на сетку субпикселей размером $\sqrt{\text{samples}} \times \sqrt{\text{samples}}$. В цикле внутри функции renderPixel генерируется несколько лучей с небольшим смещением

относительно центра пикселя. Цвета, полученные для каждого субпикселя, суммируются и усредняются. Это позволяет получить мягкие переходы на границах фигур и более высокую детализацию мелких деталей.

Описание программы

Разделение по файлам, описание основных типов данных и функций. Обязательно описать реализованные ядра.

Основные типы данных

В проекте используются специализированные структуры для эффективной передачи данных между CPU и GPU:

- **Vec3**: Базовая структура (три double или float), представляющая точку или вектор в 3D-пространстве. Оснащена методами для векторной алгебры.
- **Triangle**: Основной примитив сцены. Содержит координаты трех вершин, нормаль и цвет (в формате uchar4).
- **Camera**: Описывает положение наблюдателя (pos), точку направления взгляда (lookAt) и угол обзора (fovDeg).
- **Scene**: Легковесная структура-контейнер, передаваемая в ядро. Хранит указатель на массив треугольников в памяти GPU, их количество и параметры источников света.

Описание ключевых функций

- **renderPixel (__device__)**: Функция верхнего уровня для обработки одного пикселя. Реализует алгоритм **SSAA**. Она запускает цикл вложенных лучей со смещением субпикселей и усредняет полученные цвета для сглаживания границ.
- **shadeRay (__device__)**: Реализует модель освещения. Проверяет пересечение луча с объектами сцены, вычисляет диффузное освещение и запускает вторичные лучи (shadow rays) для проверки попадания пикселя в тень.
- **buildIcosahedron / buildDodecahedron**: Функции генерации геометрии. На основе центра и радиуса аналитически рассчитывают координаты вершин Платоновых тел и заполняют массив треугольниками.
- **computeCameraTrajectory**: Рассчитывает положение камеры для каждого кадра на основе гармонических колебаний (синусоид), обеспечивая плавное движение в анимации.

renderKernel

Тип: __global__ **Назначение:** Распределение вычислительной нагрузки по сетке потоков.

- **Логика работы:** Каждый поток вычисляет свои глобальные координаты (x,y) в сетке. Если координаты входят в границы изображения, поток рассчитывает индекс в массиве пикселей и вызывает функцию renderPixel.
- **Особенность:** Реализован "Y-flip" (инверсия по вертикали) для корректного отображения координат при записи в бинарный файл.
- **Конфигурация:** Используется строго заданная сетка блоков dim3(38, 38) с размером блока dim3(16, 16), что обеспечивает покрытие области 608×608 пикселей.

```
__global__ void renderKernel(const Camera cam,
                             int width, int height,
                             int sqrtSamples,
                             Scene scene,
                             uchar4 *outImage) {
    const int ix = blockIdx.x * blockDim.x + threadIdx.x;
    const int iy = blockIdx.y * blockDim.y + threadIdx.y;

    if (ix >= width || iy >= height) return;

    const int idx = (height - 1 - iy) * width + ix;

    outImage[idx] = renderPixel(ix, iy, width, height, sqrtSamples, cam, scene);
}
```

Покадровый рендеринг и создание анимации

Процесс генерации финального видео разделен на два этапа:

Этап 1: Рендеринг (CUDA)

Программа запускает цикл по количеству кадров (например, 100). Для каждого кадра:

1. Вычисляются новые координаты камеры.
2. Ядро GPU заполняет буфер изображения.
3. Данные копируются с GPU на CPU.
4. Кадр сохраняется в файл .data (собственный бинарный формат: width, height, массив uchar4).

Этап 2: Сборка анимации (FFmpeg)

После получения набора файлов используется утилита ffmpeg (или библиотека OpenCV) для склейки кадров.

- **Кодек:** H.264 (libx264).
- **Формат пикселя:** yuv420p (для совместимости с браузерами).
- **Процесс:** Бинарные данные считываются, преобразуются в стандартный цветовой поток и упаковываются в контейнер .mp4. Это решает проблему совместимости MIME-типов, возникающую при попытке просмотра сырых данных в браузере.

Результаты

1. Отобразить в виде таблички или графиков замеры времени работы ядер с различными конфигурациями (начиная с <<< 1, 32 >>> и как минимум до <<< 1024, 1024 >>>, для ЛР с MPI с различным числом процессов) и различными входными данными (небольшие тесты, средние и предельные).
Обязательно указать единицы измерения времени.

Единицы измерения времени в сравнениях - ms.

1. Конфигурация block(8, 8), grid(16, 16) - среднее время кадра 14.228544 ms
2. Конфигурация block(16, 16), grid(38,38) - среднее время кадра 15.226624 ms
3. Конфигурация block(32, 32), grid(4, 4) - среднее время кадра 12.834624 ms.
4. Конфигурация block(32, 8), grid(8, 8) - среднее время кадра 10.672256 ms.
5. CPU - 143.226624 ms

Параллельная GPU версия быстрее CPU по меньшей мере в 15 раз, поскольку рендеринг каждого пикселя - независимая задача, идеально подходящая для массового параллелизма GPU.

Кадры при входных данных:

```
void values_for_report() {
    std::cout << "100\n";
    std::cout << "res/%d.data\n";
    std::cout << "600 600 120\n\n";

    std::cout << "10.0 3.0 0.0  4.0 1.0  2.0 6.0 1.0  0.0 0.0\n";
    std::cout << "2.0 0.0 0.0  0.5 0.1  1.0 4.0 1.0  0.0 0.0\n\n";

    // Цвета фигур
    // Тетраэдр
    std::cout << "3.0 3.0 0.5  0.9 0.5 0.6  2.0  0.0 0.0  0\n";
    // Икосаэдр
    std::cout << "0.0 0.0 0.7  0.4 0.8 0.6  1.75  0.0 0.0  0\n";
    // Додекаэдр
```

```
std::cout << "-3.0 -3.0 0.0 0.2 0.2 0.5  1.2  0.0 0.0  0\n\n";

// Цвет пола
std::cout << "-40.0 -40.0 -1.0 -40.0 40.0 -1.0  40.0 40.0 -1.0  40.0 -40.0 -1.0 "
            "~/floor.data  0.15 0.15 0.15  0.0\n\n";
std::cout << "1\n";
std::cout << "-10.0 0.0 15.0  1.0 0.95 0.8\n\n";

std::cout << "1 4\n";
}
```

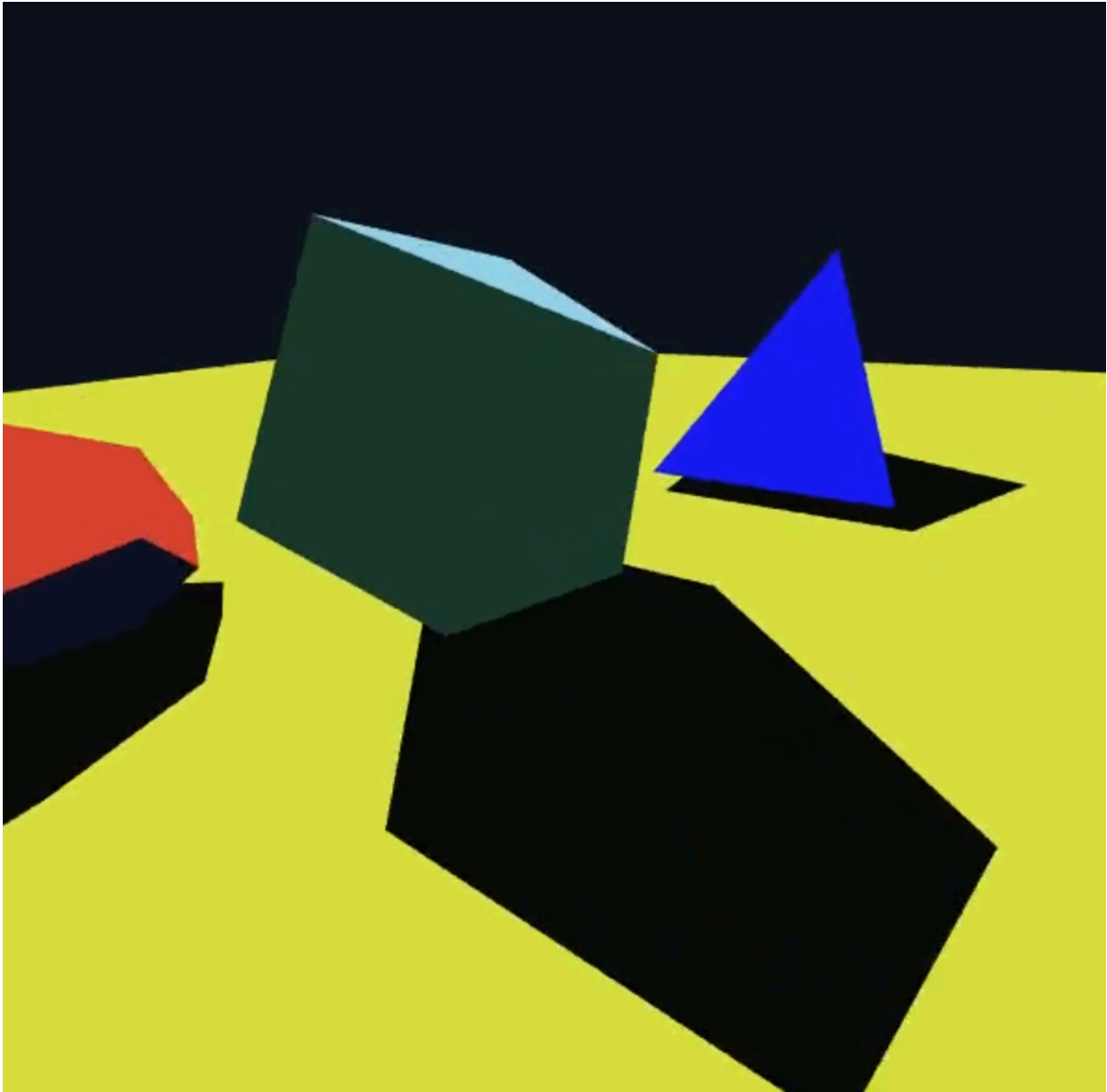


Рис. 1 — Сцена, вид сбоку.

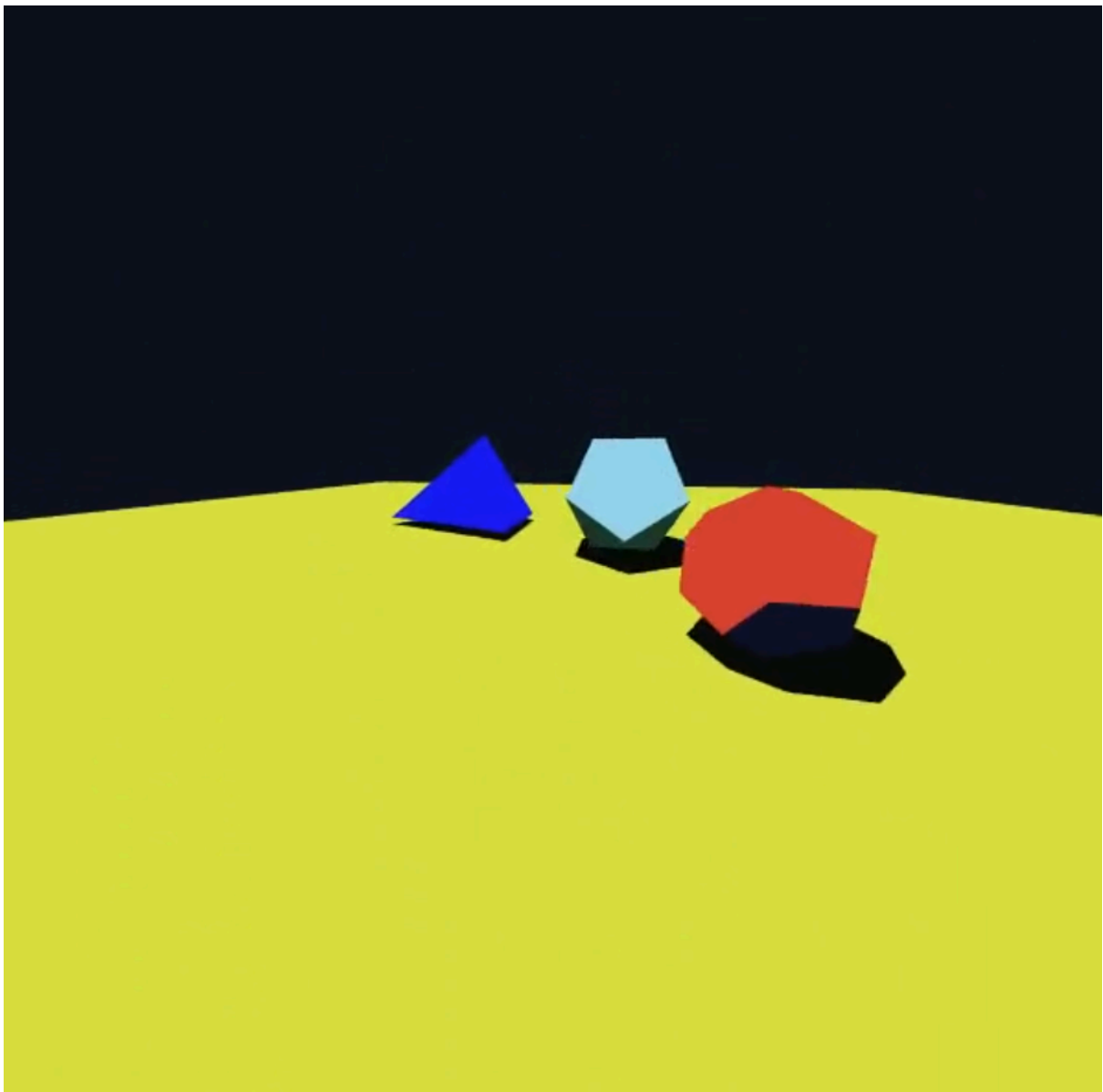


Рис. 2 — Сцена, вид спереди.

Выводы

Результаты показывают, что использование технологии CUDA является обязательным для алгоритмов трассировки лучей. Полученная анимация демонстрирует плавное движение камеры и отсутствие «зубчатости» на краях объектов благодаря SSAA, что подтверждает эффективность выбранных методов решения.

Рекурсивная трассировка лучей может применяться в компьютерной графике и анимации.